FUND NAV FETCHER

Obtención y almacenamiento de valores liquidativos

Fondos de inversión

Memoria técnica

Objetivo, arquitectura, modelo de datos y despliegue

Documento en evolución

Índice general

1. Introducción **1**

2. Arquitectura de la solución **2**

3. Arquitectura de software **3**

3.1. Responsabilidades del acceso a datos 3

4. Base de datos **4**

4.1. Bases de datos y esquemas 4

4.2. Usuarios 4

5. Modelo de datos **5**

5.1. Vista de conjunto 5

5.2. Tabla `funds` 6

5.3. Tabla `data_sources` 7

5.4. Tabla `fund_sources` 8

5.5. Tabla `fund_navs` 9

6. Despliegue a producción **10**

6.1. Construcción y publicación 10

6.2. Actualización y ejecución del servicio 10

CAPÍTULO 1

Introducción

El proyecto tiene como objetivo obtener y almacenar los valores liquidativos (NAV, por sus siglas en inglés) de fondos de inversión.

La información se recogerá de fuentes de Internet y se guardará para su consulta posterior.

Este documento recogerá progresivamente el diseño y las decisiones del proyecto.

CAPÍTULO 2

Arquitectura de la solución

Se desarrollará un servicio Python que quedará desplegado en un contenedor Docker. Consultará fuentes de Internet, pendientes de definir, para obtener los valores liquidativos de los fondos y almacenarlos en una instancia existente de PostgreSQL.

La estructura se recoge en el diagrama de arquitectura en Draw.io, disponible en `design/arquitectura.drawio`.

CAPÍTULO 3

Arquitectura de software

La arquitectura de la aplicación describe sus componentes internos y sus responsabilidades. La aplicación accede a los datos a través de un único servicio, `DataService`; no selecciona repositorios ni conoce tablas.

3.1 Responsabilidades del acceso a datos

`DataService`. Expone operaciones completas a la aplicación, comprueba las reglas de aplicación, coordina los repositorios implicados y decide cuándo confirmar (*commit*) o revertir (*rollback*). Traduce los errores de persistencia a resultados o excepciones comprensibles para sus consumidores.

`Repository` y sus repositorios concretos. La interfaz común fija el criterio compartido. Cada repositorio conoce su tabla, columnas, claves y correspondencia con los modelos; realiza las consultas de su responsabilidad y devuelve modelos o resultados definidos. No confirma transacciones ni llama a otros repositorios.

`Postgres`. Gestiona la conexión y ejecuta las operaciones contra PostgreSQL. Proporciona una única conexión transaccional compartida por los repositorios participantes y realiza el *commit* o el *rollback* que solicita `DataService`. No contiene reglas de aplicación ni conoce las entidades concretas.

Cada operación de `DataService` que modifica una o varias tablas utiliza una sola conexión a PostgreSQL, compartida por todos los repositorios que participan en ella. Así, sus consultas pertenecen a la misma transacción: `DataService` solicita el *commit* si todas terminan correctamente y el *rollback* si alguna falla. Se evita confirmar una parte de la operación y dejar el resto sin aplicar. Las claves foráneas y demás restricciones de integridad siguen siendo responsabilidad de PostgreSQL.

Los repositorios construyen las consultas específicas de sus tablas. Cuando una operación sea común y genérica, `Postgres` puede ofrecer una primitiva reutilizable para ejecutarla. Cada repositorio solicita directamente a `Postgres` que ejecute sus consultas; `DataService` no transporta SQL ni resultados del controlador.

Los repositorios no se llaman entre sí. Si una escritura referencia otra fila, no se consulta previamente su existencia solo para repetir la garantía de la clave foránea: PostgreSQL rechaza la escritura si la referencia no existe. El repositorio traduce las violaciones conocidas a errores comprensibles y `DataService` revierte la transacción y comunica el fallo. Las reglas de negocio que no cubren las restricciones referenciales, como exigir que una fuente esté habilitada, sí se comprueban desde `DataService` mediante los repositorios necesarios.

La composición configura las dependencias y credenciales al iniciar la aplicación; ese detalle no se expone a la aplicación ni a las reglas de negocio.

CAPÍTULO 4

Base de datos

Se utiliza PostgreSQL gestionado en el repositorio infra-postgresql-terraform. El aprovisionamiento de las bases de datos, los esquemas y los usuarios corresponde a esa infraestructura; este proyecto consume los recursos que se le proporcionan.

4.1 Bases de datos y esquemas

Elemento	Desarrollo	Producción
Base de datos	`psql_ddbb_dev_fund_nav`	`psql_ddbb_pro_fund_nav`
Esquema	`nav`	`nav`

El esquema `nav` se crea fuera de este proyecto y está disponible antes de desplegar la aplicación. La aplicación no crea ni elimina esquemas.

4.2 Usuarios

Uso previsto	Desarrollo	Producción
DDL	`psql_user_dev_fund_nav_deploy`	`psql_user_pro_fund_nav_deploy`
DML	`psql_user_dev_fund_nav_bot`	`psql_user_pro_fund_nav_bot`

La infraestructura proporciona ambos usuarios para cada entorno. El usuario DDL puede crear, modificar y eliminar objetos dentro del esquema asignado, pero no crear ni eliminar esquemas. El usuario DML puede consultar y modificar los datos de las tablas del esquema asignado (`SELECT`, `INSERT`, `UPDATE` y `DELETE`).

CAPÍTULO 5

Modelo de datos

El modelo registra los fondos que se desean seguir, las fuentes consultables y los valores liquidativos recogidos. La unidad del proyecto es el fondo identificado por su ISIN: cada ISIN corresponde a un registro en `funds`.

5.1 Vista de conjunto

El modelo está compuesto por cuatro tablas:

Tabla	Responsabilidad
`funds`	Catálogo de fondos seguidos, identificados por ISIN.
`data_sources`	Proveedores de datos y código del adaptador que los consulta.
`fund_sources`	Asociación de un fondo con un proveedor y los datos necesarios para consultarlo.
`fund_navs`	Histórico de valores liquidativos, fechas de valoración y procedencia.

Un fondo y un proveedor pueden existir independientemente. La tabla `fund_sources` los relaciona y permite configurar más de una fuente para un mismo fondo. Cada valor almacenado en `fund_navs` referencia esa asociación, de modo que queda registrada la procedencia del dato.

La clave primaria de `fund_navs` es `(fund_id,nav_date)`; por tanto, el diseño conserva un único valor liquidativo por fondo y fecha. La clave foránea compuesta `(fund_id,fund_source_id)` apunta a `fund_sources(fund_id,id)`. Además de comprobar que la asociación exista, impide atribuir un valor a una fuente configurada para otro fondo.

El diagrama entidad-relación está disponible en `data_model/modelo_datos.drawio`.

5.2 Tabla `funds`

Contiene los fondos que la aplicación debe seguir. El ISIN es obligatorio y único, y constituye el identificador externo del fondo; la clave primaria interna `id` se utiliza en las relaciones entre tablas. Un fondo puede registrarse antes de configurar sus fuentes.

Los fondos nunca se eliminan de `funds`. Cuando se deja de seguir un fondo, se desactiva estableciendo `funds.active=false`. Esta operación detiene la recogida de nuevos valores liquidativos y conserva el registro del fondo, sus asociaciones en `fund_sources` y su histórico en `fund_navs`. Si se vuelve a seguir el fondo, se reactiva el mismo registro mediante `funds.active=true`; la recogida requiere también una asociación y un proveedor activos.

Los identificadores internos de los ejemplos son ilustrativos.

Campo	Tipo	Ejemplo	Clave	Descripción
`id`	bigint	42	PK	Identificador interno generado por PostgreSQL.
`isin`	varchar(12)	`ES0152769032`	UQ, NOT NULL	ISIN único que identifica el fondo.
`name`	text	Fondo Naranja S&P 500 FI	NOT NULL	Nombre del fondo.
`currency`	varchar(3)	`EUR`	NOT NULL	Divisa en la que se expresa el NAV.
`active`	boolean	`true`	NOT NULL	Indica si se recogen nuevos valores.

5.3 Tabla `data_sources`

Define cada proveedor de valores liquidativos de forma independiente de los fondos que comercializa. El campo `code` identifica el adaptador Python que implementa la consulta.

Los proveedores nunca se eliminan de `data_sources`. Cuando se deja de consultar un proveedor, se desactiva estableciendo `data_sources.active=false`. Esta operación detiene las consultas a ese proveedor y conserva su registro, sus asociaciones en `fund_sources` y el histórico de valores liquidativos que aportó en `fund_navs`, incluida su procedencia. Si se vuelve a consultar el proveedor, se reactiva el mismo registro mediante `data_sources.active=true`; la recogida requiere también un fondo y una asociación activos.

Campo	Tipo	Ejemplo	Clave	Descripción
`id`	bigint	3	PK	Identificador interno generado por PostgreSQL.
`code`	text	`amundi`	UQ, NOT NULL	Código estable del adaptador Python.
`name`	text	Amundi	NOT NULL	Nombre legible del proveedor.
`base_url`	text	`https://amundi.es`	NOT NULL	Dirección base pública, sin credenciales.
`active`	boolean	`true`	NOT NULL	Permite detener consultas conservando el histórico.

5.4 Tabla `fund_sources`

Relaciona un fondo con un proveedor y almacena la configuración específica necesaria para consultar sus valores. Cada pareja fondo-proveedor es única. Los campos `external_id` y `nav_url` son opcionales: se usan cuando el proveedor requiere un identificador distinto del ISIN o una dirección específica.

Campo	Tipo	Ejemplo	Clave	Descripción
`id`	bigint	12	PK	Identificador de esta asociación.
`fund_id`	bigint	42	FK, NOT NULL	Referencia `funds(id)`.
`source_id`	bigint	3	FK, NOT NULL	Referencia `data_sources(id)`.
`external_id`	text	`FND-12345`	Opcional	Identificador del fondo en el proveedor, si hace falta.
`nav_url`	text	`https://datos.example/f/12345`	Opcional	URL específica de consulta para los valores del fondo.
`priority`	integer	1	NOT NULL	Un número menor tiene preferencia.
`active`	boolean	`true`	NOT NULL	Habilita o deshabilita esta ruta.

La restricción `UNIQUE(fund_id,source_id)` evita duplicar la asociación para el mismo proveedor. La restricción `UNIQUE(fund_id,id)` permite que `fund_navs` compruebe mediante una FK compuesta que la fuente pertenece al fondo indicado.

5.5 Tabla `fund_navs`

Almacena el valor liquidativo seleccionado para cada fondo y fecha, junto con el momento de recogida y la asociación de proveedor que lo originó. La fecha es la fecha de valoración publicada por la fuente, no necesariamente el día en que se descargó el dato.

Campo	Tipo	Ejemplo	Clave	Descripción
`fund_id`	bigint	42	PK parcial, FK compuesta	Identifica el fondo.
`nav_date`	date	`2026-10-08`	PK parcial	Fecha de valoración.
`nav_value`	numeric(80,48)	`45.6705203`	NOT NULL	Valor liquidativo positivo, en la divisa del fondo.
`fund_source_id`	bigint	12	FK compuesta, NOT NULL	Asociación del fondo y proveedor que originó el dato.
`fetched_at`	timestamptz	`2026-10-08 18:30+00`	NOT NULL	Instante de recogida, por defecto la hora actual.

La FK compuesta `(fund_id,fund_source_id)` referencia `fund_sources(fund_id,id)`. Así, la base de datos rechaza tanto una asociación inexistente como una asociación que pertenezca a otro fondo. El modelo conserva un único valor por fondo y fecha; no guarda observaciones simultáneas de varios proveedores ni versiones anteriores de correcciones.

CAPÍTULO 6

Despliegue a producción

La aplicación se ejecuta como un servicio Docker en el servidor personal *Bigoci*. Su despliegue se automatiza con GitHub Actions y utiliza Docker Hub como registro de imágenes, desde el que el servidor descarga la versión que debe ejecutar.

6.1 Construcción y publicación

Cada actualización de la rama `main` activa el flujo de GitHub Actions. Primero se construye una imagen para Linux ARM64, basada en Python 3.13, que incluye la aplicación y sus dependencias de ejecución. La imagen se publica en Docker Hub como `serhuela/fund-nav-fetcher:latest`.

Docker Hub almacena la imagen preparada para su ejecución; el despliegue consumirá esa imagen sin volver a construirla. La etiqueta `latest` se actualiza con cada publicación.

6.2 Actualización y ejecución del servicio

Una vez publicada la imagen, GitHub Actions copia la configuración de Docker Compose al servidor personal *Bigoci* y accede a él mediante SSH. Descarga la imagen y recrea el contenedor con Docker Compose.

El contenedor arranca con `APP_ENV=production` y ejecuta la aplicación con un usuario sin privilegios de administrador. Docker Compose configura el reinicio automático del servicio, salvo que se haya detenido expresamente, y limita los registros a veinte archivos de cinco megabytes cada uno.